Lecture 40 Practice Solutions (Hinglish): Deep Learning (Part-5)

Complete Mathematical Derivations & Code Solutions

Solution 1: CIFAR-10 Normalization Mechanics

1. **Mathematical Derivation:** `transforms.ToTensor()` raw pixels $[0, 255]$ ko $[0.0, 1.0]$ me convert karta hai ($x = p/255.0$). `transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))` formula apply karta hai:

$$x_{\text{norm}} = \frac{x - \mu}{\sigma} = \frac{x - 0.5}{0.5} = 2x - 1$$

   - Minimum value ($x = 0.0$): $2(0) - 1 = -1.0$
   - Midpoint ($x = 0.5$): $2(0.5) - 1 = 0.0$
   - Maximum value ($x = 1.0$): $2(1) - 1 = +1.0$ Isliye normalized data exact $[-1.0, 1.0]$ range me zero-centered ho jata hai.

2. **Zero-Centered Data kyu zaroori hai:** Agar input x_i hamesha positive rahega ($[0, 1]$), toh weights ka gradient $\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial z} x_i$ hamesha same sign ka hoga. Isse saare weights ek saath badhenge ya ek saath ghatenge, jisse gradient descent zigzag path follow karega aur convergence bohot slow ho jayega. Zero-centering se yeh issue solve hota hai.

Solution 2: Feature Map Spatial Shape Calculation

Formula: $H_{\text{out}} = \lfloor \frac{H_{\text{in}} - K + 2P}{S} \rfloor + 1$. $K = 3, P = 1, S = 1$ par height/width same rehti hai ($32 \to 32$). MaxPool $(2, 2)$ spatial size ko half karta hai ($32 \to 16$).

- **Input:** `(Batch, 3, 32, 32)`
- **Block 1:** Conv2d ($3 \to 32$) $\to$ `(Batch, 32, 32, 32)` $\to$ MaxPool $\to$ `(Batch, 32, 16, 16)`
- **Block 2:** Conv2d ($32 \to 64$) $\to$ `(Batch, 64, 16, 16)` $\to$ MaxPool $\to$ `(Batch, 64, 8, 8)`
- **Block 3:** Conv2d ($64 \to 128$) $\to$ `(Batch, 128, 8, 8)` $\to$ MaxPool $\to$ `(Batch, 128, 4, 4)`

- **Flattening:**

$$\text{Nodes} = 128 \times 4 \times 4 = \mathbf{2,048} \text{ features}$$

Solution 3: Parameter Count Analysis

Formulas:

- Conv: $(K \times K \times C_{\text{in}} + 1) \times C_{\text{out}}$
- Linear: $(\text{in} + 1) \times \text{out}$

1. **Conv1 (3 to 32, 3x3):** $(3 \times 3 \times 3 + 1) \times 32 = 28 \times 32 = \mathbf{896}$
2. **Conv2 (32 to 64, 3x3):** $(3 \times 3 \times 32 + 1) \times 64 = 289 \times 64 = \mathbf{18,496}$
3. **Conv3 (64 to 128, 3x3):** $(3 \times 3 \times 64 + 1) \times 128 = 577 \times 128 = \mathbf{73,856}$
4. **Linear1 (2048 to 256):** $(2048 + 1) \times 256 = 2049 \times 256 = \mathbf{524,544}$
5. **Linear2 (256 to 10):** $(256 + 1) \times 10 = 257 \times 10 = \mathbf{2,570}$
6. **Total Parameters:**

$$896 + 18,496 + 73,856 + 524,544 + 2,570 = \mathbf{620,362} \text{ parameter}$$

Solution 4: Model.eval() aur torch.no_grad()

1. `model.eval()` :
    - **Dropout:** Training me random neurons drop hote the, testing me dropout OFF ho jata hai aur saare neurons full active rehte hain.
    - **BatchNorm:** Training me batch statistics update hoti hain, testing me running mean aur variance freeze ho jaate hain.
2. `with torch.no_grad()` :
    - PyTorch autograd engine computation graph banana band kar deta hai. Agar ye na lagayein, toh testing ka har intermediate tensor RAM/VRAM me store hota rahega aur Out-Of-Memory error aa jayega.

Solution 5: VGG Factorization & Receptive Field

1. Receptive field: $RF = RF_{\text{prev}} + (K - 1)$.
    - Filter 1 (3×3): $RF_1 = 1 + 2 = 3$.

- Filter 2 (3×3): $RF_2 = 3 + 2 = 5$. Do 3×3 layers milkar exact 5×5 ka field cover karti hain.

2. Parameter Ratio:
   - Single 5×5: $25C^2$.
   - Two 3×3: $2 \times 9C^2 = 18C^2$.

$$\frac{18C^2}{25C^2} = 0.72 \implies \textbf{28\%} \text{ parameters kam lagte hain!}$$

Solution 6: ResNet Residual Highway

1. $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$
2. Derivative:

$$\frac{\partial \mathcal{E}}{\partial \mathbf{x}} = \frac{\partial \mathcal{E}}{\partial \mathbf{y}} \left(\frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \mathbf{I} \right) = \frac{\partial \mathcal{E}}{\partial \mathbf{y}} \frac{\partial \mathcal{F}}{\partial \mathbf{x}} + \frac{\partial \mathcal{E}}{\partial \mathbf{y}}$$

3. Identity matrix $\mathbf{I}$ ki wajah se term $\frac{\partial \mathcal{E}}{\partial \mathbf{y}}$ direct bina kisi multiplication ke seedha pichli layer tak flow karta hai. Chahe $\mathcal{F}$ zero ho jaye, gradient kabhi 0 nahi banta.

Solution 7: Stemming vs Lemmatization

1. Outputs:
   - `"running"` : Stemming → `"run"` , Lemmatization → `"run"`
   - `"better"` : Stemming → `"better"` , Lemmatization → `"good"`
   - `"flies"` : Stemming → `"fli"` , Lemmatization → `"fly"`
   - `"corpora"` : Stemming → `"corpora"` , Lemmatization → `"corpus"`
2. **Over-stemming:** Alag words ko ek hi galat stem me kaat dena (jaise `"universal"` aur `"university"` dono ko `"univers"` bana dena). **Under-stemming:** Same root wale words ko match na kar pana (jaise `"alumnus"` aur `"alumni"` alag reh jana).

Solution 8: One-Hot Encoding ki Limitations

1. Memory: $500 \text{ words} \times 100,000 \text{ dimensions} = 50,000,000 \text{ floats}$.
   $50,000,000 \times 4 \text{ bytes} \approx \textbf{190.73}$ MB sirf ek document ke liye!
2. Orthogonality: Distinct words ke one-hot vectors ka dot product zero hota hai: $\mathbf{e}_k \cdot \mathbf{e}_q = 0 \implies \cos\theta = 0$. Is wajah se "king" aur "queen" me model zero similarity

dekhta hai.

Solution 9: Bag-of-Words DTM

1. Sorted Vocabulary: `['deep', 'learn', 'learning', 'models', 'neural', 'patterns']`
2. Matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 1 \\ 2 & 1 & 0 & 1 & 1 & 1 \end{bmatrix}$$

3. OOV: `"transformers generate text"` ka koi bhi word vocabulary me nahi hai, toh iska vector all-zero $[0, 0, 0, 0, 0, 0]$ banega.

Solution 10: N-Grams se Negation Handling

1. BoW me dono sentences ke word counts identical hain ($[1, 1, 1, 1, 2]$), isliye linear model dono ko same predict karega.
2. Bigrams: S_1 me `"not good"` aur `"was bad"` aayega. S_2 me `"not bad"` aur `"was good"` aayega. Bigram se negation capture ho jata hai.

Solution 11: TF-IDF Calculation

1. $\text{TF}(\text{"cat"}, D_2) = \frac{3}{3} = \mathbf{1.0}$, $\text{TF}(\text{"cat"}, D_1) = \frac{1}{3} \approx \mathbf{0.3333}$.
2. $\text{IDF}(\text{"cat"}) = \log_{10}(4/3) \approx \mathbf{0.1249}$, $\text{IDF}(\text{"mat"}) = \log_{10}(4/1) \approx \mathbf{0.6021}$.
3. $\text{TF-IDF}(\text{"cat"}, D_2) = 1.0 \times 0.1249 = \mathbf{0.1249}$. $\text{TF-IDF}(\text{"mat"}, D_1) = 0.3333 \times 0.6021 = \mathbf{0.2007}$. Rare word "mat" ko higher importance mili!

Solution 12: Word2Vec Math

1. CBOW context words se center word predict karta hai. Skip-Gram center word se context words predict karta hai.
2. $\mathbf{u} \cdot \mathbf{v} = 0.8(0.6) + 0.6(0.8) + 0 = 0.96$. Dono unit vectors hain ($\|\mathbf{u}\| = 1, \|\mathbf{v}\| = 1$).

$$\cos(\theta) = \frac{0.96}{1 \times 1} = \mathbf{0.96}$$

3. Gender relation vector space me ek direction shift ke barabar hota hai ($\vec{v}_{\text{Woman}} - \vec{v}_{\text{Man}}$), jisko King me add karne se wo Queen ke paas shift ho jata hai.

Solution 13: RNN Parameter Count

1. Shapes & Params:
   - $\mathbf{W}_{xh}(256 \times 100) = 25,600$
   - $\mathbf{W}_{hh}(256 \times 256) = 65,536$
   - $\mathbf{b}_h(256) = 256$
   - $\mathbf{W}_{hy}(10 \times 256) = 2,560$
   - $\mathbf{b}_y(10) = 10$
2. Total = $\mathbf{93,962}$ parameters.
3. Sequence length $T = 10$ se $T = 1,000$ hone par parameters **same rahenge** kyunki parameter sharing ki wajah se har step par wahi same weight matrices multiply hoti hain.

Solution 14: BPTT Vanishing Gradients & Clipping

1. BPTT me $\mathbf{W}_{hh}^T$ matrix $t - k$ times continuously multiply hoti hai, jo exponential decay ya explosion create karti hai.
2. $\tanh'(z) \leq 1.0$ hota hai, aur aksar $0.2 - 0.4$ hota hai. Lagataar 10 steps multiply hone par gradient zero ban jata hai.
3. Gradient Clipping:

$$\mathbf{g}_{\text{clipped}} = \theta \frac{\mathbf{g}}{\|\mathbf{g}\|} = 5.0 \frac{\mathbf{g}}{25.0} = \mathbf{0.2} \cdot \mathbf{g}$$

   Vector ki direction same rehti hai aur magnitude 5.0 ho jata hai.

Solution 15: LSTM Cell Gates & CEC Proof

1. 4 Steps:
   - Forget Gate: $\mathbf{f}_t = \sigma(\mathbf{W}_f \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$
   - Input Gate: $\mathbf{i}_t = \sigma(\mathbf{W}_i \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$, $\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_c \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c)$
   - Cell State: $\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$
   - Output Gate: $\mathbf{o}_t = \sigma(\mathbf{W}_o \cdot [\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$, $\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$

2. Derivative along the highway:

$$\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} \approx \mathbf{f}_t$$

3. Agar network forget gate ko open rakhta hai ($\mathbf{f}_t \approx 1$), toh gradient product $1.0 \times 1.0 \cdots = 1.0$ bana rehta hai, jisse gradient bina decay hue hazaaron time steps tak travel kar sakta hai (Constant Error Carousel).